

Compiler Design and Language Processors

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Compiler Design and Language Processors — Complete Notes	3
1. Introduction to Language Processors	3
1.1 Language Processing System	3
1.2 Assemblers	4
1.3 Interpreters	5
1.4 Loaders and Linkers	5
1.5 Macro Processing	6
2. Compiler Structure and Phases	6
2.1 Phases of a Compiler	6
2.2 Passes of a Compiler	7
2.3 Compiler Construction Tools	7
3. Lexical Analysis	7
3.1 Role of the Lexical Analyzer	7
3.2 Input Buffering	8
3.3 Specification and Recognition of Tokens	8
4. Syntax Analysis (Parsing)	8
4.1 Grammars	8
4.2 Syntax Error Handling	9
4.3 Top-Down Parsing	9
4.4 Bottom-Up Parsing	10
5. Semantic Analysis and Intermediate Code	11
5.1 Semantic Analysis	11
5.2 Intermediate Code Representation	12
6. Storage Organization	12
6.1 Run-Time Memory Layout	12
6.2 Activation Record (Stack Frame)	13
6.3 Allocation Strategies	13
6.4 Parameter-Passing Methods	13
6.5 Scope and Symbol Table Organization	13
7. Code Optimization and Code Generation	13
7.1 Code Optimization	13
7.2 Code Generation	14
8. Quick Revision and Exam Pointers	14

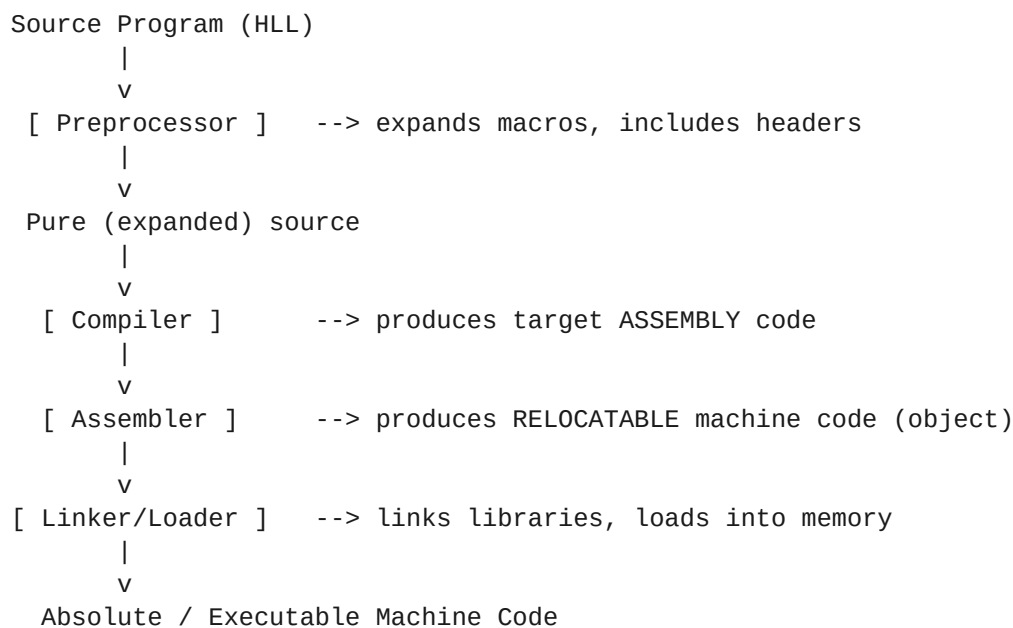
Compiler Design and Language Processors — Complete Notes

This guide builds from the basics of language processing to advanced parsing, optimization, and code generation, written for the **IBPS SO (IT Officer)** MCQ exam. Every keyword that an examiner can ask is highlighted.

1. Introduction to Language Processors

1.1 Language Processing System

A **language processor** is software that translates or executes a program written in a programming language. A typical processing pipeline transforms a high-level **source program** into an executable **machine-code** program.



1.1.1 Translator and Preprocessor

A **translator** converts a program from one language to another. The three classical translators are the **compiler**, the **interpreter**, and the **assembler**.

A **preprocessor** is a program that processes source text BEFORE compilation. Its jobs include **macro expansion**, **file inclusion** (**#include**), **conditional compilation** (**#ifdef**), and **rational preprocessing** (adding extra control-flow facilities).

1.1.2 Compiler vs Interpreter vs Assembler

A **compiler** translates an entire high-level program into machine/target code in one shot and reports all errors together; the produced object code runs **faster** because translation happens **only once**.

An **interpreter** reads, translates, and executes the program **statement by statement**; it stops at the first error, making debugging easier but execution **slower** (no object code is stored).

An **assembler** is a special translator that converts **assembly language** (mnemonics) into **machine code** (object code).

Feature	Compiler	Interpreter	Assembler
Input	High-level language	High-level language	Assembly language
Output	Object/target code	No object code (direct execution)	Machine code
Translation unit	Whole program at once	One statement at a time	Whole program
Execution speed	Faster	Slower	Fast

Feature	Compiler	Interpreter	Assembler
Error reporting	All at once	One at a time (stops first)	All at once
Memory	Needs space for object code	Less (no object file)	Object file
Examples	C, C++, Java(javac)	Python, BASIC, Ruby	MASM, NASM

MCQ Trap: An interpreter does **NOT** produce object/machine code; it executes directly. Compilers are faster at run time but slower to debug. Java is **both** — **javac** compiles to **bytecode**, then the **JVM** interprets/JIT-compiles it.

Memory Trick: "Compiler = Complete first, run later; Interpreter = Immediate line-by-line."

1.2 Assemblers

1.2.1 Functions of an Assembler

The **assembler** performs these core tasks:

- Convert **mnemonic opcodes** to their **machine (binary) opcodes** using the **OPTAB**.
- Convert symbolic operands (labels) to **machine addresses** using the **SYMTAB**.
- Maintain a **Location Counter (LC)** to assign addresses sequentially.
- Process **pseudo-operations / assembler directives** (e.g., **START**, **END**, **RESW**, **BYTE**, **EQU**).
- Build the **literal table (LITTAB)** and generate the **object program**.

1.2.2 Single-Pass Assembler

A **single-pass (one-pass) assembler** scans the source **only once** and generates code immediately. The challenge is the **forward reference** problem (a symbol used before it is defined); it is solved by **backpatching** — leaving the address blank and filling it later once the symbol is defined.

1.2.3 Two-Pass Assembler

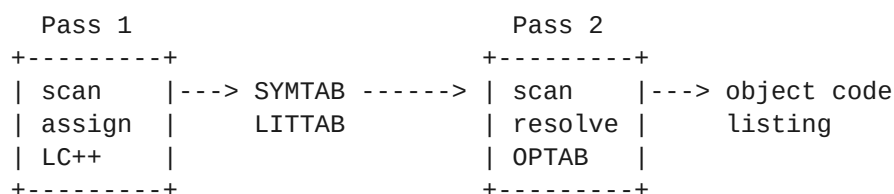
A **two-pass assembler** scans the source **twice**:

- **Pass 1 (Define):** scan all statements, build the **SYMTAB** (assign an address to every label via the **LC**), build the **LITTAB**, and process the **START/EQU** directives. No object code yet.
- **Pass 2 (Generate):** translate each statement to **object code**, resolve operands using **OPTAB** + **SYMTAB**, and emit the **object program** (records: **Header H**, **Text T**, **Modification M**, **End E**).

MCQ Trap: The **forward reference** problem is naturally solved by the **two-pass** assembler (symbols are all known after Pass 1). A **single-pass** assembler needs **backpatching** to handle forward references.

1.2.4 Assembler Data Structures (Tables)

Table	Full form	Purpose	Built/Used
OPTAB	Opcode/Machine-op table	Mnemonic to binary opcode + instruction length	Used in both passes
SYMTAB	Symbol table	Label to address mapping	Built Pass 1 , used Pass 2
LITTAB	Literal table	Literals and their assigned addresses	Pass 1/Pass 2
POT	Pseudo-op table	Directives (START , END , ...)	Both passes



1.3 Interpreters

An **interpreter** directly executes the source program without producing a separate machine-code file. It re-translates a statement **every time** it runs (e.g., a statement inside a loop is decoded on each iteration).

- **Advantages: easier debugging** (immediate error at the line), **machine independence/portability**, smaller memory footprint (no object file), good for rapid development and scripting.
- **Disadvantages: slower execution**, source must be present at run time, repeated re-translation of loop bodies.

Shortcut: Use an interpreter when you value **portability + debugging**; use a compiler when you value **run-time speed**.

1.4 Loaders and Linkers

1.4.1 Loader Functions

A **loader** is a system program that places an object program into memory and prepares it for execution. Its four classic functions are **Allocation** (reserve memory), **Linking** (resolve external symbol references), **Relocation** (adjust address-dependent locations), and **Loading** (place code + data into memory).

Loader type	Linking	Relocation	Key idea
Absolute loader	No (done by programmer/assembler)	No	Loads at a fixed address; simplest & fastest
Bootstrap loader	No	No	First loader run at power-on ; loads the OS
Relocating (BSS) loader	No	Yes	Uses relocation bits ; load anywhere
Direct linking loader	Yes	Yes	General; uses ESD, RLD, TXT, END records
Dynamic loading	At call time	—	A routine is loaded only when called

A **bootstrap loader** is the small, often ROM-resident loader that brings the operating system into memory when the computer is switched on.

Dynamic loading loads a subroutine into memory **only when it is invoked**, improving memory utilization (unused routines are never loaded).

1.4.2 Linker

A **linker (linkage editor)** combines several **object modules** and library routines into a single **executable/load module**, resolving **external references** (a symbol defined in one module and used in another).

Aspect	Static linking	Dynamic linking
When	At link/compile time	At load/run time
Library code	Copied into the executable	Shared (DLL / .so)
Executable size	Larger	Smaller
Memory sharing	No (each program has a copy)	Yes (one copy shared)
Update libraries	Must re-link	Replace shared lib; no re-link

MCQ Trap: Linking = combine modules + resolve external references; **Loading** = bring the program into memory for execution. **Relocation** can be done by the assembler, linker, OR loader.

Memory Trick: Loader's four jobs = "**A-L-R-L**" = **A**llocation, **L**inking, **R**elocation, **L**oading.

1.5 Macro Processing

A **macro** is a single-line abbreviation for a group of statements; the **macro processor** replaces each **macro call** with the corresponding statement sequence (**macro expansion**) BEFORE/at assembly time.

- **Macro definition:** enclosed between **MACRO** and **MEND** directives.
- **Macro call (invocation):** the macro name appears in the program.
- **Macro expansion:** the call is replaced **in-line** by the macro body, with **arguments substituted** for formal parameters.

A **macro processor** uses three databases:

- **MNT** — **Macro Name Table** (names + index into MDT).
- **MDT** — **Macro Definition Table** (the stored body lines).
- **ALA** — **Argument List Array** (maps formal parameters to actual arguments).

Nested macro: a macro **call inside another macro definition**. **Conditional macro expansion:** uses **AIF** (conditional branch), **AGO** (unconditional branch), and **SET** symbols to generate different code per call (conditional assembly).

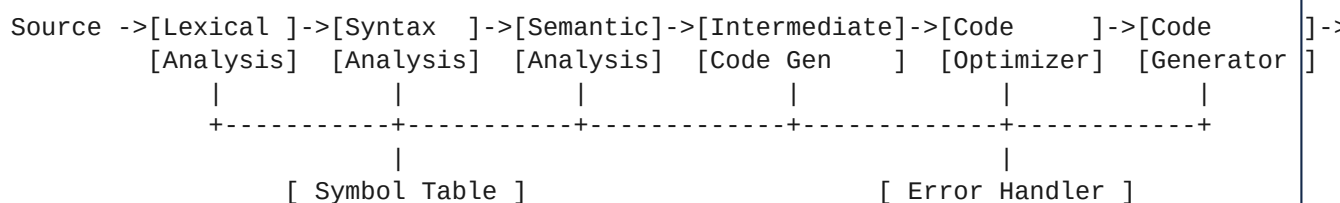
Feature	Macro	Subroutine
Code copies	Replicated at each call (in-line)	Single copy
Control transfer	None (expanded in place)	CALL/RETURN
Object code size	Larger	Smaller
Execution speed	Faster (no call overhead)	Slower (call/return)

MCQ Trap: A **macro** is expanded **in-line** (more memory, no transfer of control), whereas a **subroutine** transfers control (less memory, more overhead). Macros trade **space** for **time**.

2. Compiler Structure and Phases

2.1 Phases of a Compiler

A compiler is organized into **six phases**, supported by the **Symbol Table Manager** and the **Error Handler** which interact with all phases.



#	Phase	Input	Output
1	Lexical Analysis (Scanner)	Character stream	Tokens
2	Syntax Analysis (Parser)	Tokens	Parse tree / syntax tree
3	Semantic Analysis	Syntax tree	Annotated (checked) tree
4	Intermediate Code Generation	Annotated tree	Three-address code
5	Code Optimization	Intermediate code	Optimized IR
6	Code Generation	Optimized IR	Target/machine code

The compiler is split into two logical halves:

- **Analysis phase (Front End):** **lexical, syntax, semantic** analysis, intermediate code generation, and machine-**independent** optimization. It is **source-language dependent** and target-machine **independent**.

- **Synthesis phase (Back End):** machine-**dependent** optimization and **code generation**. It is **target-machine dependent**.

MCQ Trap: Lexical analysis removes white-space and comments; the parser does **NOT** see them. **Type checking** happens in **Semantic analysis**, NOT syntax analysis. The order is fixed: **Lexical -> Syntax -> Semantic -> ICG -> Optimize -> Code Gen**.

Memory Trick: Phase order = "**Long Snakes Sometimes Internally Optimize Code**" = **Lexical**, **Syntax**, **Semantic**, **ICG**, **Optimize**, **Code-gen**.

2.2 Passes of a Compiler

A **pass** is one complete traversal of the source/intermediate representation.

- **Single-pass compiler:** all phases handled in **one** traversal — **fast, less memory**, but limited optimization (e.g., classic Pascal).
- **Multi-pass compiler:** uses **multiple** traversals; enables better optimization and complex languages but is **slower** and needs more memory.

MCQ Trap: Do not confuse **phase** (a logical sub-task like parsing) with **pass** (a physical traversal). Several phases can be grouped into one pass.

2.3 Compiler Construction Tools

Tool	Category	Produces
LEX / Flex	Scanner generator	Lexical analyzer from regular expressions (yylex)
YACC / Bison	Parser generator	LALR(1) parser from a CFG (yparse)
Syntax-directed translation engines	Translation	Code from parse-tree traversals
Automatic code generators	Code gen	Target code from IR templates
Data-flow analysis engines	Optimization	Information for global optimization

Shortcut: **LEX** = **tokens (RE based)**, **YACC** = **grammar/parsing (LALR(1) based)**. YACC literally means "**Yet Another Compiler-Compiler**."

3. Lexical Analysis

3.1 Role of the Lexical Analyzer

The **lexical analyzer (scanner)** reads the source character stream and groups characters into **tokens**. It also **strips comments and white-space**, inserts identifiers into the **symbol table**, and correlates errors with **line numbers**. The parser calls it via **getNextToken()**.

- **Token:** a category/class with a name and optional attribute, e.g., **id**, **num**, **keyword**, **operator**.
Form = **<token-name, attribute>**.
- **Pattern:** the **rule** (usually a **regular expression**) describing the set of strings for a token.
- **Lexeme:** the **actual character sequence** in the source matching a pattern.

```

      getNextToken()
Source --> [ Lexical ] -----> [ Parser ]
      chars   [ Analyzer ] <--token--
              |      ^
              v      | (insert / lookup)
              [ Symbol Table ]

```

MCQ Trap: Token vs Lexeme vs Pattern — for **int count;**, the **lexeme count** matches the **pattern** for identifiers and is classified as the **token id**. Many lexemes can map to **one** token.

3.2 Input Buffering

To read the source efficiently, the scanner uses **buffer pairs** and **sentinels**.

- **Buffer pairs:** two buffer halves of **N characters** each; when one half is exhausted, the other is reloaded. Two pointers — **lexemeBegin** and **forward** — delimit the current lexeme.
- **Sentinel:** a special **eof** character placed at the **end of each half** so a single test detects both buffer-end and input-end, reducing the number of comparisons per character.

```

| lexeme ... | eof | ... second half ... | eof |
  ^           ^
lexemeBegin forward

```

3.3 Specification and Recognition of Tokens

3.3.1 Regular Expressions and Definitions

Tokens are specified using **regular expressions (RE)**. The three operations, in **decreasing precedence**, are **Kleene star *** (highest), then **concatenation**, then **union |** (lowest).

- `letter = [A-Za-z_], digit = [0-9]`
- `id = letter (letter | digit)*`
- `num = digit+ ( . digit+ )? ( E [+ -]? digit+ )?`

3.3.2 Finite Automata and Conversions

A **transition diagram** is a hand-built recognizer; formally tokens are recognized by **finite automata**.

- **NFA (Non-deterministic FA):** allows **ϵ -moves** and multiple transitions on one symbol.
- **DFA (Deterministic FA):** exactly **one** transition per symbol, **no** ϵ -moves — directly usable as code.

The standard tool chain is **RE -> NFA -> DFA -> minimized DFA**:

Step	Algorithm	Note
RE to NFA	Thompson's construction	Adds ϵ -transitions
NFA to DFA	Subset construction	Worst case 2^n states
DFA minimization	Partitioning / Hopcroft	$O(n \log n)$ (Hopcroft)

RE --Thompson--> NFA --Subset--> DFA --Minimize--> Minimal DFA

MCQ Trap: NFA and DFA accept exactly the same class of languages (regular languages) — NFA is **not** more powerful, only more compact. The subset construction can blow up to 2^n DFA states in the worst case.

4. Syntax Analysis (Parsing)

4.1 Grammars

A **Context-Free Grammar (CFG)** is a 4-tuple **$G = (V, T, P, S)$** where **V** = non-terminals, **T** = terminals, **P** = productions, **S** = start symbol.

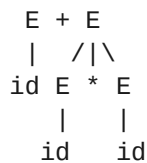
- **Derivation:** repeatedly replacing a non-terminal by a production body. A **leftmost derivation (LMD)** always expands the **leftmost** non-terminal; a **rightmost derivation (RMD / canonical)** expands the **rightmost** one.
- **Parse tree:** a tree whose **leaves are terminals** and **internal nodes are non-terminals**; the yield (left-to-right leaves) is the derived string.

A grammar is **ambiguous** if some string has **more than one parse tree** (or more than one leftmost / rightmost derivation).

```

      E          E -> E + E | E * E | id
    / | \

```

String: id + id * id
(an ambiguous grammar yields two distinct parse trees)

4.1.1 Left Recursion and Left Factoring

- **Left recursion** $A \rightarrow A \alpha \mid \beta$ must be removed for top-down parsing. **Elimination:** $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$.
- **Left factoring** removes a common prefix: $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ becomes $A \rightarrow \alpha A'$ and $A' \rightarrow \beta_1 \mid \beta_2$ (needed to make a grammar **LL(1)**).

4.1.2 Chomsky Hierarchy

Type	Grammar	Language	Recognizer
Type 0	Unrestricted	Recursively enumerable	Turing Machine
Type 1	Context-sensitive	Context-sensitive	Linear Bounded Automaton
Type 2	Context-free	Context-free	Pushdown Automaton (PDA)
Type 3	Regular	Regular	Finite Automaton

MCQ Trap: $\text{Type 3} \subset \text{Type 2} \subset \text{Type 1} \subset \text{Type 0}$ (regular is the most restricted).
Programming-language **syntax** is **Type 2 (CFG)**; **tokens** are **Type 3 (regular)**.

4.2 Syntax Error Handling

Four **error-recovery strategies**:

- **Panic-mode recovery:** discard input tokens until a **synchronizing token** (e.g., `;`, `}`) is found — simplest, never loops, may skip a lot.
- **Phrase-level recovery:** perform a **local correction** on the remaining input (e.g., insert a missing `;`).
- **Error productions:** augment the grammar with rules for **common errors**.
- **Global correction:** find the **minimum number of changes** to fix the program — theoretically optimal but **costly** (Burke-Fisher / least-cost).

4.3 Top-Down Parsing

Top-down parsing builds the tree from the **root (start symbol)** down to the leaves, tracing a **leftmost derivation**.

4.3.1 Recursive-Descent and Backtracking

Recursive-descent parsing uses one (possibly recursive) procedure per non-terminal. A naive version may need **backtracking** (try, fail, retry) — slow and unable to handle **left recursion**.

4.3.2 Predictive / LL(1) Parsing

A **predictive parser** is a backtrack-free, table-driven recursive descent parser. **LL(1)** = scan **Left-to-right**, produce **Leftmost** derivation, with **1** symbol of lookahead. It needs the grammar to be **non-left-recursive** and **left-factored** and uses a parsing table **M[A, a]**.

A grammar is **LL(1)** iff for every pair of productions $A \rightarrow \alpha \mid \beta$:

- **FIRST**(α) $\cap$ **FIRST**(β) = $\emptyset$, and
- if $\epsilon \in \text{FIRST}(\beta)$ then **FIRST**(α) $\cap$ **FOLLOW**(A) = $\emptyset$.

4.3.3 Computing FIRST and FOLLOW

FIRST(X) = set of terminals that begin strings derivable from X:

- If X is a terminal, $\text{FIRST}(X) = \{X\}$.
- If $X \rightarrow \epsilon$, add ϵ to $\text{FIRST}(X)$.
- If $X \rightarrow Y_1 Y_2 \dots Y_k$, add $\text{FIRST}(Y_1) - \{\epsilon\}$; if $\epsilon \in \text{FIRST}(Y_1)$ continue to Y_2 , and so on; add ϵ only if **all** Y_i derive ϵ .

FOLLOW(A) = set of terminals that can appear **immediately to the right** of A :

- Put $\$$ (end-marker) in **FOLLOW(S)** (start symbol).
- For $A \rightarrow \alpha B \beta$, add $\text{FIRST}(\beta) - \{\epsilon\}$ to **FOLLOW(B)**.
- For $A \rightarrow \alpha B$ (or $A \rightarrow \alpha B \beta$ with $\epsilon \in \text{FIRST}(\beta)$), add **FOLLOW(A)** to **FOLLOW(B)**.

Worked example for the classic expression grammar $E \rightarrow T E' \mid E' \rightarrow + T E' \mid \epsilon \mid T \rightarrow F T' \mid T' \rightarrow * F T' \mid \epsilon \mid F \rightarrow (E) \mid \text{id}$:

Symbol	FIRST	FOLLOW
E	{ (, id }	{) , \$ }
E'	{ + , ϵ }	{) , \$ }
T	{ (, id }	{ + ,) , \$ }
T'	{ * , ϵ }	{ + ,) , \$ }
F	{ (, id }	{ + , * ,) , \$ }

MCQ Trap: ϵ is never placed in any FOLLOW set; $\$$ is never in any FIRST set (it is added only to FOLLOW). A grammar that is **left-recursive or ambiguous can never be LL(1)**.

4.4 Bottom-Up Parsing

Bottom-up parsing builds the tree from the **leaves up to the root**, tracing a **rightmost derivation in reverse**. The general class is **shift-reduce parsing**.

4.4.1 Handle and Handle Pruning

A **handle** is a substring that matches the **right side of a production** and whose **reduction** is one step of the reverse rightmost derivation. **Handle pruning** is the repeated process of locating a handle and **reducing** it; this is the essence of bottom-up parsing.

4.4.2 Shift-Reduce Parsing

A **shift-reduce parser** uses a **stack** and an input buffer with four actions:

- **Shift:** push the next input symbol onto the stack.
- **Reduce:** replace a handle on top of the stack by its non-terminal.
- **Accept:** parsing succeeds.
- **Error:** a syntax error is discovered.

Stack	Input	Action
\$	id+id*id\$	shift
\$id	+id*id\$	reduce $E \rightarrow id$
\$E	+id*id\$	shift ...
...		...
\$E	\$	ACCEPT

4.4.3 Operator-Precedence Parsing

For an **operator grammar** (no two adjacent non-terminals, no ϵ -productions) the parser uses three **precedence relations** — $<\cdot$ (**yields precedence**), $=\cdot$ (**equal**), and $\cdot>$ (**takes precedence**) — to find handles.

4.4.4 LR Parsing Family

LR(k) = scan **L**eft-to-right, build **R**ightmost derivation in reverse, with **k** lookahead. The parser is driven by an **ACTION** table and a **GOTO** table built from the **canonical collection of LR items** over the **augmented grammar** $S' \rightarrow S$.

Stack(states/symbols) | Input | ACTION/GOTO tables --> Driver --> shift/reduce/accept

Parser	Item / lookahead	Reduction decided by	#States	Power
LR(0)	LR(0) items, no lookahead	Reduce on every state	base	Weakest
SLR(1)	LR(0) items	FOLLOW set	= LR(0)	Weak
LALR(1)	merged LR(1) cores	per-state lookahead	= SLR	Medium
CLR / LR(1)	full LR(1) items	per-state lookahead	Most	Strongest

Key relationships an examiner loves:

- **Power (grammars accepted):** $LR(0) \subset SLR(1) \subset LALR(1) \subset CLR(1) = LR(1)$.
- **Number of states:** $SLR = LALR < CLR$ (SLR and LALR have the **SAME** number of states; CLR has the most because cores are not merged).
- Every **LL(1)** grammar is **LR(1)**, but not vice-versa.
- **LALR** is built by **merging LR(1) states with the same core (LR(0) items)**.

4.4.5 Parsing Conflicts

- **Shift-reduce (S-R) conflict:** parser cannot decide whether to shift or reduce; classic example is the **dangling-else**, resolved by **preferring shift** (else binds to nearest if).
- **Reduce-reduce (R-R) conflict:** two reductions are possible in one state.

MCQ Trap: State merging in LALR can introduce a **NEW** reduce-reduce conflict that CLR did not have, but it **never** introduces a new **shift-reduce** conflict. **YACC** builds **LALR(1)** parsers.

Memory Trick: Power order "**baby Steps Lead Climbers up**" = $LR(0) < SLR < LALR < CLR$.

5. Semantic Analysis and Intermediate Code

5.1 Semantic Analysis

Semantic analysis checks meaning: **type checking**, **scope resolution**, **declaration-before-use**, and array-bound/argument consistency. It is driven by **syntax-directed** machinery.

- **Syntax-Directed Definition (SDD):** a CFG + **attributes** on grammar symbols + **semantic rules**.
- **Syntax-Directed Translation (SDT):** an SDD with **semantic actions** embedded in the production bodies $\{ \dots \}$.

Attribute	Computed from	Evaluation pass	Parser fit
Synthesized	children (and the node)	bottom-up	LR / S-attributed
Inherited	parent + siblings	top-down / left-to-right	LL / L-attributed

- An **S-attributed** definition uses **only synthesized** attributes and can be evaluated **bottom-up** in a single pass.
- An **L-attributed** definition allows inherited attributes that depend only on the **parent and left siblings**, evaluated in **one left-to-right pass**.

MCQ Trap: A leaf/terminal can have a **synthesized** attribute but **never** an **inherited** one. **Every S-attributed grammar is also L-attributed**, but not the reverse.

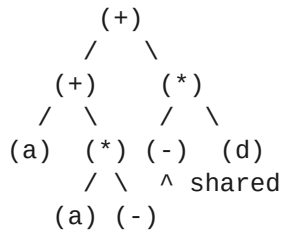
5.2 Intermediate Code Representation

5.2.1 Syntax Tree and DAG

An **Abstract Syntax Tree (AST)** is a condensed parse tree where operators are **internal nodes** and operands are **leaves**; non-essential symbols (like parentheses) are dropped.

A **Directed Acyclic Graph (DAG)** is like an AST but **shares common subexpressions** (a repeated computation has a single node), making it ideal for detecting redundancy.

$a + a * (b - c) + (b - c) * d$ (the two $(b - c)$ share ONE node in the DAG)



5.2.2 Three-Address Code (TAC)

Three-address code has at most **one operator** on the right and at most **three addresses** per instruction: $x = y \text{ op } z$. It uses **temporaries** ($t1, t2, \dots$). Three implementations exist:

Representation	Structure	Reordering for optimization
Quadruples	(op, arg1, arg2, result)	Easy (result names are explicit)
Triples	(op, arg1, arg2); result = position	Hard (positions shift)
Indirect triples	list of pointers to triples	Easy (reorder the pointer list)

For $a + b * c$:

Quadruples	Triples
(*, b, c, t1)	(0) (*, b, c)
(+, a, t1, t2)	(1) (+, a, (0))

5.2.3 Postfix Notation

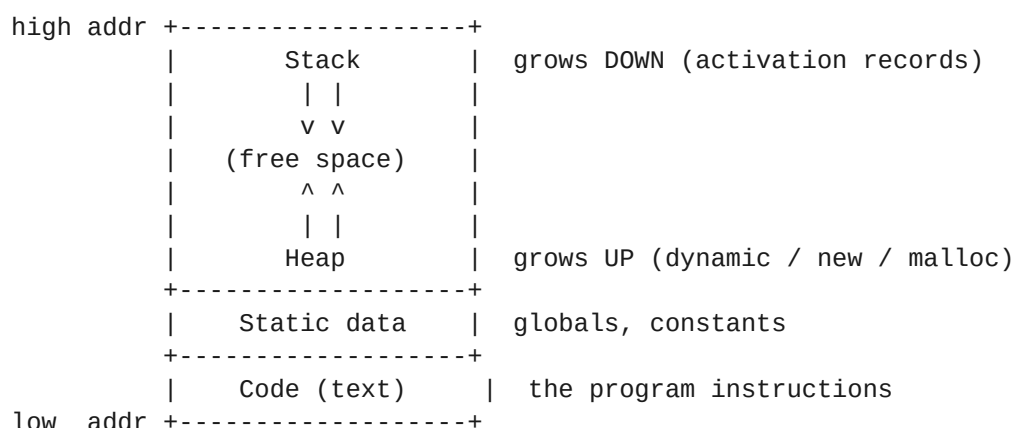
Postfix (Reverse Polish) notation writes the operator **after** its operands and needs **no parentheses**; e.g., $a + b * c$ becomes $a \ b \ c \ * \ +$.

MCQ Trap: **Quadruples** are easiest to **move/reorder** during optimization; **triples** save memory but are **hard to reorder** because they reference results by **position**. **Indirect triples** fix this with a pointer list.

6. Storage Organization

6.1 Run-Time Memory Layout

At run time, memory is divided into four regions:



- **Code area:** fixed-size, holds the target program.
- **Static area:** globals and data whose size is known at **compile time**.
- **Stack:** holds **activation records** for procedure calls (LIFO).
- **Heap:** **dynamically** allocated data that may outlive the call that created it; managed by allocator + **garbage collection**.

6.2 Activation Record (Stack Frame)

An **activation record** is the block of information for **one procedure call**:

+-----+	
Return value	
Actual parameters	
Control link (dynamic)	--> caller's activation record
Access link (static)	--> for non-local data access
Saved machine status	(return address, saved registers)
Local data	
Temporaries	
+-----+	

6.3 Allocation Strategies

Strategy	Allocated	Supports recursion?	Lifetime
Static	Compile time	No	Whole program
Stack	At call (push/pop)	Yes	Per activation
Heap	On demand (run time)	Yes	Until freed / GC

6.4 Parameter-Passing Methods

Method	What is passed	Caller sees changes?
Call by value	A copy of the value	No
Call by reference	The address (l-value)	Yes (immediately)
Call by value-result (copy-restore)	Copy in, copy back at return	Yes (only at return)
Call by name	Textual substitution (re-evaluated)	Yes (lazy, Algol-60)

MCQ Trap: **Call by reference** changes the actual argument **immediately**; **call by value-result** copies back **only on return** — results differ when the same variable is passed and also accessed globally (aliasing).

6.5 Scope and Symbol Table Organization

The **symbol table** stores each identifier with attributes (type, scope, size, memory location). Nested scopes are handled by a **stack of hash tables** or by **most-closely-nested scope rules**; a block's table is pushed on entry and popped on exit.

7. Code Optimization and Code Generation

7.1 Code Optimization

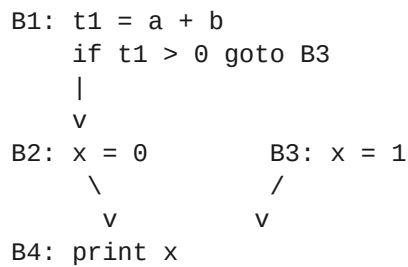
Optimization improves code **without changing its meaning**, targeting speed or size.

- **Machine-independent** optimization works on the **intermediate code**.
- **Machine-dependent** optimization works on the **target code** (registers, addressing modes).

7.1.1 Basic Blocks and Flow Graphs

A **basic block** is a maximal sequence of instructions with a **single entry (the leader)** and **single exit**, no jumps in/out except at the ends.

Leaders are: (1) the **first statement**, (2) any **target of a jump**, and (3) any statement **immediately following a jump**. A **flow graph** has basic blocks as nodes and control-flow as directed edges.



7.1.2 Optimization Techniques

Technique	What it does	Example
Constant folding	Evaluate constants at compile time	$x = 3 * 4$ to $x = 12$
Constant propagation	Replace variable by known constant	use 12 where x is 12
Common subexpression elimination	Reuse a computed value	compute $a+b$ once
Dead-code elimination	Remove never-used code	drop unused $x = 5$
Copy propagation	Replace x by y after $x = y$	use y directly
Strength reduction	Replace costly op by cheaper	$x*2$ to $x+x$ / shift
Loop-invariant code motion	Move unchanging code out of a loop	hoist $y = a+b$
Induction-variable elimination	Simplify loop counters	combine i and j

- **Local optimization:** within a **single basic block**.
- **Global optimization:** **across** basic blocks, using **data-flow analysis**.

7.1.3 Peephole Optimization

Peephole optimization scans a **small sliding window (peephole)** of target instructions and replaces them with shorter/faster equivalents. Techniques: **redundant load/store elimination**, **flow-of-control optimization** (jumps to jumps), **algebraic simplification** ($x = x + 0$), **strength reduction**, and **use of machine idioms** (auto-increment).

7.2 Code Generation

The **code generator** is the final phase; it maps optimized intermediate code to **target machine code**.

Issues in code generation:

- **Input** form (IR) and **target program** form (absolute / relocatable / assembly).
- **Instruction selection** (choose the best machine instructions).
- **Register allocation** (decide **which** variables live in registers) and **register assignment** (decide the **specific** register) — optimal allocation is **NP-complete**.
- **Evaluation order** of computations.

A simple code generator uses a `getReg()` function plus **next-use information** and an **address/register descriptor** to emit instructions.

MCQ Trap: Register allocation = *which* values go in registers; **register assignment** = *which specific* register. Optimal register allocation is **NP-complete**. Code generation is **machine-dependent** (back end), so it is **not** portable across architectures.

8. Quick Revision and Exam Pointers

- **Compiler phase order:** **Lexical -> Syntax -> Semantic -> Intermediate -> Optimization -> Code Generation**; symbol table + error handler span all.

- **Front end** = analysis (source-dependent); **Back end** = synthesis (machine-dependent).
- **Lexical** produces **tokens**; **Syntax** produces a **parse tree**; **Semantic** does **type checking**.
- **FIRST/FOLLOW**: \$ only in FOLLOW; ϵ never in FOLLOW.
- **LL(1)** is **top-down/predictive**; **SLR, LALR, CLR** are **bottom-up/LR**.
- **Power**: $LR(0) < SLR(1) < LALR(1) < CLR(1)$; **#states**: $SLR = LALR < CLR$.
- **YACC** = **LALR(1)**, **LEX** = **RE-based scanner**.
- **Two-pass assembler** solves **forward references**; **single-pass** uses **backpatching**.
- **Linker** resolves **external references**; **Loader** brings code into memory.
- **TAC forms**: **quadruples** (easy reorder), **triples** (position-based), **indirect triples** (pointer list).
- **NFA and DFA** recognize the **same** (regular) languages; **subset construction** can give 2^n states.

Memory Trick: Confusing **handle**? A **handle** is what you **reduce** in **bottom-up** parsing — "**handle** for **harvesting** **bottom-up**."

Shortcut: Whenever a question says "type checking", the answer is **Semantic Analysis**; "remove comments/white-space" is **Lexical Analysis**; "**intermediate code with temporaries**" is **Three-Address Code****.